

CourseFlow

API Reference

Generated 07 September 2026

Source docs/API.md

Next.js 16.3.0

Surface 26 route handler files · 42 endpoints · 0 server actions

Contents

1. Overview	6
2. Authentication & authorization	7
Session establishment	7
Roles	7
How handlers check auth	7
Route protection (proxy)	8
Rate limiting	8
3. Endpoints	10
3.1 Catalog	10
GET /api/courses	10
3.2 Auth	12
POST /api/auth/register	12
POST /api/auth/forgot-password	13
GET /auth/confirm	15
3.3 Profile	16
PATCH /api/profile	16
POST /api/profile/avatar	17
3.4 Enrollments	18
GET /api/enrollments	18
POST /api/enrollments	19
3.5 Wishlist	20
GET /api/wishlist?count=1	20
POST /api/wishlist	20
DELETE /api/wishlist	21
3.6 Progress	22
POST /api/progress	22
3.7 Assignment submissions	23
PUT /api/assignments/{id}/submission	23
POST /api/assignments/{id}/submission/file	25

3.8 Promo codes	26
POST /api/promo-codes/validate	26
3.9 Payments	28
POST /api/payments/charge	28
GET /api/payments/charges/{id}	30
POST /api/payments/webhook	31
3.10 Admin — courses	31
GET /api/admin/courses	32
POST /api/admin/courses	33
GET /api/admin/courses/{id}	36
PUT /api/admin/courses/{id}	37
PATCH /api/admin/courses/{id}	37
DELETE /api/admin/courses/{id}	38
GET /api/admin/course-tags	38
3.11 Admin — lessons	38
GET /api/admin/courses/{id}/lessons	38
POST /api/admin/courses/{id}/lessons	39
GET /api/admin/courses/{id}/lessons/{lessonId}	40
PUT /api/admin/courses/{id}/lessons/{lessonId}	41
DELETE /api/admin/courses/{id}/lessons/{lessonId}	41
3.12 Admin — assignments	42
GET /api/admin/assignments	42
POST /api/admin/assignments	43
GET /api/admin/assignments/{id}	45
PATCH /api/admin/assignments/{id}	46
DELETE /api/admin/assignments/{id}	47
3.13 Admin — promo codes	47
GET /api/admin/promo-codes	47
POST /api/admin/promo-codes	48
GET /api/admin/promo-codes/{id}	50
PATCH /api/admin/promo-codes/{id}	51

<code>DELETE /api/admin/promo-codes/{id}</code>	51
3.14 Admin — uploads	51
<code>POST /api/admin/uploads</code>	51
<code>GET /api/admin/uploads/signed-url</code>	53
4. Server actions	54
5. Common error shapes	55
Status codes in this project	55
6. Known gaps	57

CourseFlow API Reference

Derived by reading the code in this repository. Every endpoint, field, and status code below is traceable to a route handler, a validation helper, or a migration in `docs/sql/`.

Two markers are used throughout:

- **(schema)** — the shape is enforced by a shared validation helper, so it is reliable.
- **(handler)** — the shape was read out of the handler body or a Supabase `select`, so it reflects what the code does today rather than a declared contract.
- *(inferred)* — marks anything not directly confirmed. All of these are collected in [Known gaps](#).

There is no Zod in this project. `package.json` has no schema-validation dependency. All request validation is hand-rolled in `src/lib/*-validation.js`. Where those helpers exist they are the source of truth and marked **(schema)**; everywhere else the shape came from reading the handler body and is marked **(handler)**.

1. Overview

CourseFlow's API surface is **26 route handler files exposing 42 method+path endpoints**. There are **no React server actions** — see [Server actions](#).

Base URL. Routes are served by the Next.js app itself; there is no separate API host. Local development is `http://localhost:3000`. All paths below are relative to that origin.

Where the code lives. 25 route files sit under `src/app/api/`. One more — `GET /auth/confirm` — sits outside that tree at `src/app/auth/confirm/route.js`, because it is a redirect target for Supabase email links rather than a JSON API.

Next.js version. This repo runs Next.js 16.3.0. Two conventions from `node_modules/next/dist/docs/` matter when reading these handlers:

- Dynamic route params arrive as a **Promise** and must be awaited: `export async function GET(request, { params }) { const { id } = await params; }`. Every dynamic handler here does this.
- A route file exports one function per HTTP method (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `HEAD`, `OPTIONS`); `OPTIONS` is auto-implemented when absent. No handler in this repo defines `HEAD` or `OPTIONS`.

Content types.

Direction	Type	Used by
Request	<code>application/json</code>	Most endpoints
Request	<code>multipart/form-data</code>	<code>POST /api/profile/avatar</code> , <code>POST /api/admin/uploads</code> , <code>POST /api/assignments/{id}/submission/file</code>
Response	<code>application/json</code>	All JSON endpoints (set explicitly by <code>jsonOk</code>)
Response	<code>307</code> redirect	<code>GET /auth/confirm</code> only

Response helpers. All JSON responses come from `src/lib/api.js`:

Helper	Produces
<code>jsonOk(data, init)</code>	<code>data</code> verbatim, status 200 unless overridden
<code>jsonError(message, status, extras)</code>	<code>{ "error": message, ...extras }</code>
<code>jsonTooManyRequests(sec, message)</code>	<code>{ "error": message }</code> plus a <code>Retry-After</code> header, status 429

There is no envelope on success — handlers return their payload at the top level.

2. Authentication & authorization

Session establishment

Auth is Supabase, cookie-based. There is **no bearer-token or API-key path** — every authenticated endpoint reads the session from cookies.

- `src/lib/supabase/server.js` → `createClient()` builds a `@supabase/ssr` server client wired to Next's `cookies()` store. Handlers call this (directly or via `src/lib/auth.js`) and the Supabase auth cookies travel on the request.
- `src/proxy.js` runs `updateSession()` on nearly every request, refreshing the session and writing rotated cookies onto the response.
- `createServiceClient()` returns a **service-role client that bypasses RLS**, built from `SUPABASE_SERVICE_ROLE_KEY`. It returns `null` when that variable is absent. It is used for the public catalog read path and the user lookup in forgot-password. Its own docblock describes it as "only for temporary local testing".

Because auth rides on cookies, browser callers need no extra headers. A non-browser client must send the Supabase auth cookies.

Row Level Security is a second enforcement layer. Most tables have RLS policies (`docs/sql/*.sql`) scoping rows to `auth.uid()`. A handler that passes its auth check can still get an empty result or an error if RLS rejects the query.

Roles

Roles live in `public.profiles`, not in the JWT.

Role	Stored as	Can do
Guest	no session	Read the public catalog only
Authenticated user	session; <code>profiles.role</code> anything but <code>admin</code>	Own enrollments, wishlist, progress, submissions, payments
Admin	<code>profiles.role = 'admin'</code> and <code>profiles.is_active = true</code>	Manage courses, lessons, assignments, promo codes, uploads

Admin requires **both** conditions. A profile with `role = 'admin'` but `is_active = false` is not treated as an admin anywhere in this codebase.

How handlers check auth

Two helpers in `src/lib/auth.js`. Both return an `error` field holding a ready-made `Response`, and every handler returns it immediately on failure.

```
const { supabase, user, error } = await requireUser();
if (error) return error;
```

Helper	Checks	Failure
<code>requireUser()</code>	A session exists	401 {"error": "Unauthorized"}
<code>requireAdmin()</code>	Session exists, then <code>role === 'admin' && is_active === true</code>	401 if no session; 403 {"error": "Forbidden"} if the session is not an admin

Exact bodies an unauthenticated or unauthorized caller receives:

```
{ "error": "Unauthorized" }
```

```
{ "error": "Forbidden" }
```

Note the ordering inside `requireAdmin`: no session yields **401**, while a valid non-admin session yields **403**.

Route protection (proxy)

`src/proxy.js` guards admin **pages** before any handler runs. Its matcher covers all paths except `_next/static`, `_next/image`, `favicon.ico`, `api/admin/uploads`, and requests ending in an image extension.

Caller at <code>/admin/*</code>	Result
No session	307 redirect to <code>/admin/login?next=<path></code>
Session, not an admin	307 redirect to <code>/</code>
Active admin	Request proceeds

This covers page routes. `/api/admin/*` handlers are **not** redirected — they enforce auth themselves through `requireAdmin()` and return JSON 401/403. The proxy comments say as much: "API handlers enforce auth via `requireAdmin()`".

Rate limiting

`src/lib/rate-limit.js` implements an in-memory sliding window. Its docblock notes it works for a single `next start` process and **does not share state across serverless replicas**.

Endpoint	Limit	Window	Keyed on
GET <code>/api/courses</code>	60	60s	client IP
GET <code>/api/admin/courses</code>	120	60s	client IP

Endpoint	Limit	Window	Keyed on
GET /api/admin/assignments	120	60s	client IP
GET /api/admin/promo-codes	120	60s	client IP
POST /api/admin/assignments	20	15 min	admin user id
POST /api/auth/register	5	15 min	client IP
POST /api/auth/forgot-password	10	60s	client IP
POST /api/auth/forgot-password	1	60s	client IP + email
POST /api/promo-codes/validate	database-side	—	see note below
POST /api/payments/charge	database-side	—	see note below

Client IP is read from `x-forwarded-for` (first entry), then `x-real-ip`, else the literal `"unknown"`.

The last two rows are **not** limited by `rate-limit.js`. They call the `lookup_checkout_promo` Postgres RPC, which applies its own shared limit; the handler forwards a `429` when the RPC reports one.

Response when tripped — status `429` with a `Retry-After` header in seconds:

```
{ "error": "Too many searches, try again in a moment" }
```

The message varies: `"Too many registration attempts, try again in a moment"`, `"Too many assignment creates, try again in a moment"`, `"Too many reset requests. Please try again shortly."`, `"Please wait before requesting another reset link."`.

3. Endpoints

3.1 Catalog

GET /api/courses

Public course catalog with search, sort, and pagination.

Auth: public. Behaviour changes when a session is present: enrolled courses are excluded from results and `enrolledCourseIds` is populated.

Query params

Name	Type	Required	Description
<code>q</code>	string	No	Search over <code>title</code> and <code>summary</code> . Max 100 chars (<code>CATALOG_SEARCH_MAX_LENGTH</code>); longer returns 400.
<code>page</code>	integer	No	1-based. Default <code>1</code> . Must be an integer ≥ 1 .
<code>pageSize</code>	integer	No	Only <code>6</code> or <code>12</code> are accepted. Any other value returns 400.
<code>sortBy</code>	string	No	<code>title</code> , <code>price</code> , <code>createdAt</code> , <code>updatedAt</code> , <code>lessonCount</code> , <code>hours</code> . Unrecognised values fall back to <code>createdAt</code> .
<code>sortDirection</code>	string	No	<code>asc</code> or <code>desc</code> . Default <code>desc</code> .
<code>includeUserState</code>	string	No	<code>"1"</code> adds <code>wishlistIds</code> to the response. Requires a session to be meaningful.

Success — 200

Field	Type	Description
<code>courses</code>	array	Course cards (below)
<code>total</code>	integer	Total matching rows before pagination
<code>enrolledCourseIds</code>	string[]	Course ids the caller is enrolled in; <code>[]</code> when logged out

Field	Type	Description
wishlistIds	string[]	Only present when includeUserState=1

Each course card (`mapCatalogCourse` , **handler**):

Field	Type	Notes
id	uuid	
code	string	course_code , falling back to id
courseCode	string	Same value as code
title	string	
summary	string	
coverUrl	string	Resolved from cover_image_url or cover_file_url
lessonCount	integer	
hours	number	Numeric total_learning_time , else 0
totalLearningTime	string	Raw column value
price	number	THB

```
{
  "courses": [
    {
      "id": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34",
      "code": "MKT101",
      "courseCode": "MKT101",
      "title": "Digital Marketing Fundamentals",
      "summary": "Build a practical foundation in channels, funnels, and campaign measurement.",
      "coverUrl": "/courses/service-design.svg",
      "lessonCount": 3,
      "hours": 10,
      "totalLearningTime": "10",
      "price": 4590
    }
  ],
  "total": 8,
  "enrolledCourseIds": []
}
```

The response carries `Cache-Control: private, no-store, max-age=0, must-revalidate`. The database result is cached server-side for logged-out callers, but the HTTP response never is.

Errors

Status	Trigger
400	<code>Invalid page or page size</code> — <code>pageSize</code> not 6/12, or <code>page</code> not an integer ≥ 1
400	<code>Search query is too long</code> — <code>q</code> over 100 characters
429	Rate limit (60/min per IP)
500	Underlying query threw; message is the thrown error's text

3.2 Auth

POST /api/auth/register

Registers a new learner through Supabase Auth. On success the handler immediately signs the new session out, so registering does **not** log the caller in.

Auth: public.

Request body — validated by `validateAll()` in `src/lib/register-validation.js` (**schema**)

Field	Type	Required	Rules
<code>fullName</code>	string	Yes	Non-empty
<code>dob</code>	string (<code>YYYY-MM-DD</code>)	Yes	Valid date, not in the future, age ≥ 18
<code>education</code>	string	No	Never rejected — its validator always returns <code>""</code>
<code>email</code>	string	Yes	Must match <code>/^[^\s@]+@[^\s@]+\.[^\s@]+\$/</code>
<code>password</code>	string	Yes	At least 6 characters
<code>confirmPassword</code>	string	Yes	Must equal <code>password</code>

```
{
  "fullName": "Napat Chaiyaporn",
  "dob": "1996-04-12",
  "education": "Bachelor of Business Administration",
  "email": "napat.c@example.com",
  "password": "correct-horse-battery",
  "confirmPassword": "correct-horse-battery"
}
```

Success — 200

```
{ "ok": true }
```

Errors

Status	Body	Trigger
400	<code>{"error": "Invalid JSON body"}</code>	Body is unparseable, or not a non-array object
400	<code>{"error": "Please check the required registration fields", "errors": {...}}</code>	Field validation failed; <code>errors</code> maps every field name to a message (empty string when valid)
400	<code>{"error": "<supabase message>"}</code>	Supabase <code>signUp</code> rejected it (e.g. email already registered)
429	Rate limit	5 per 15 min per IP

```
{
  "error": "Please check the required registration fields",
  "errors": {
    "fullName": "",
    "dob": "You must be at least 18 years old",
    "education": "",
    "email": "",
    "password": "Password must be at least 6 characters",
    "confirmPassword": "Passwords do not match"
  }
}
```

POST /api/auth/forgot-password

Sends a Supabase password-reset email.

Auth: public.

Request body (handler)

Field	Type	Required	Description
email	string	Yes	Must match the email regex. Lower-cased and trimmed before use.
redirectTo	string	No	Absolute <code>http / https</code> URL. Invalid values are ignored and the <code>Origin</code> header is used instead.

```
{
  "email": "napat.c@example.com",
  "redirectTo": "http://localhost:3000/reset-password"
}
```

Success — 200

Field	Type	Description
ok	boolean	Always <code>true</code>
email	string	Normalised email
cooldownSec	integer	<code>60</code> — the per-email window in seconds

```
{ "ok": true, "email": "napat.c@example.com", "cooldownSec": 60 }
```

Errors

Status	Body	Trigger
400	<code>{"error": "Invalid JSON body"}</code>	Unparseable body
400	<code>{"error": "Please enter a valid email."}</code>	Missing or malformed email
400	<code>{"error": "Unable to determine password reset redirect URL."}</code>	No usable <code>redirectTo</code> and no <code>Origin</code> header
404	<code>{"error": "No account found with this email. Please check and try again."}</code>	Email not in Supabase Auth
429	Rate limited	10/min per IP, or 1/min per IP+email, or Supabase itself returned 429

Status	Body	Trigger
500	<code>{"error": "Password reset is not configured."}</code>	<code>SUPABASE_SERVICE_ROLE_KEY</code> missing — <code>createServiceClient()</code> returned <code>null</code>

This endpoint **confirms whether an email is registered** (404 vs 200). That is an account-enumeration signal; noted here as observed behaviour, not a recommendation.

The registered-email lookup pages through `auth.admin.listUsers()` at 200 per page, up to 20 pages — so it only sees the first ~4,000 users.

GET /auth/confirm

Redirect target for Supabase email links (password recovery and similar). Returns redirects, never JSON. Located at `src/app/auth/confirm/route.js`, outside `app/api/`.

Auth: public — the link's token is the credential.

Query params

Name	Type	Required	Description
<code>token_hash</code>	string	With <code>type</code>	Verified server-side via <code>verifyOtp</code>
<code>type</code>	string	With <code>token_hash</code>	Supabase OTP type, e.g. <code>recovery</code>
<code>code</code>	string	Alternative	PKCE code; forwarded to the client for exchange
<code>next</code>	string	No	Post-verification path. Must start with <code>/</code> and not <code>//</code> , else the default reset path is used.
<code>error</code>	string	No	Passed through from Supabase
<code>error_description</code>	string	No	Passed through from Supabase

Behaviour

Case	Result
<code>token_hash</code> + <code>type</code> verify successfully	<code>307</code> to <code>next</code> , with session cookies set
<code>code</code> only	<code>307</code> to <code>next</code> with <code>?code=...</code> — the browser holds the PKCE verifier and completes the exchange

Case	Result
<code>error</code> present, verification fails, neither credential supplied, or auth not configured	<code>307</code> to the reset path with <code>?error=access_denied&error_description=...</code>

3.3 Profile

PATCH /api/profile

Updates the caller's own profile, and their auth email if it changed.

Auth: authenticated.

Request body — reuses `validateAll()` but only enforces `fullName`, `dob`, `email` (**schema**)

Field	Type	Required	Rules
<code>fullName</code>	string	Yes	Non-empty
<code>dob</code>	string (<code>YYYY-MM-DD</code>)	Yes	Valid date, not future, age $\geq$ 18
<code>education</code>	string	No	Stored as <code>educational_background</code>
<code>email</code>	string	No	Defaults to the session email. If different, <code>supabase.auth.updateUser</code> is called.
<code>removeAvatar</code>	boolean	No	Exactly <code>true</code> sets <code>avatar_url</code> to <code>null</code>

```
{
  "fullName": "Napat Chaiyaporn",
  "dob": "1996-04-12",
  "education": "Bachelor of Business Administration",
  "email": "napat.c@example.com",
  "removeAvatar": false
}
```

Success — `200`

```
{
  "profile": {
    "id": "3c9a7e21-5d84-4f0b-8a16-7be2c4d09f55",
    "full_name": "Napat Chaiyaporn",
    "date_of_birth": "1996-04-12",
    "educational_background": "Bachelor of Business Administration",
    "avatar_url": "https://<project>.supabase.co/storage/v1/object/public/profile-avatars/3c9a7e21-.../a1b2.jpg"
  }
}
```

Errors

Status	Body	Trigger
400	<code>{"error": "Invalid JSON body"}</code>	Unparseable body
400	<code>{"error": "Please check the required profile fields", "errors": {...}}</code>	<code>fullName</code> / <code>dob</code> / <code>email</code> invalid
400	<code>{"error": "<msg>", "errors": {"email": "<msg>"}}</code>	Supabase rejected the email change
401	<code>{"error": "Unauthorized"}</code>	No session
500	<code>{"error": "Failed to update profile"}</code>	Profile update failed

POST /api/profile/avatar

Uploads an avatar to the `profile-avatars` bucket and writes the public URL onto the profile.

Auth: authenticated.

Request: `multipart/form-data`

Field	Type	Required	Rules
<code>file</code>	File	Yes	<code>image/jpeg</code> or <code>image/png</code> only; max 5 MB; non-empty

Stored at `{userId}/{uuid}.{jpg|png}` with `upsert: false`.

Success — 200

```
{ "avatarUrl": "https://<project>.supabase.co/storage/v1/object/public/profile-avatars/3c9a7e21-.../a1b2.jpg" }
```

Errors

Status	Body	Trigger
400	<code>{"error": "Expected multipart form data"}</code>	Body is not form data
400	<code>{"error": "Photo is required"}</code>	No file, no name, or zero bytes
400	<code>{"error": "Only JPG and PNG photos are supported"}</code>	Other MIME type
400	<code>{"error": "Photo must be 5 MB or smaller"}</code>	Over 5 MB
401	<code>{"error": "Unauthorized"}</code>	No session
500	<code>{"error": "Failed to upload photo"} / {"error": "Failed to save photo"}</code>	Storage or profile write failed. On profile-write failure the uploaded object is deleted again.

3.4 Enrollments

GET /api/enrollments

Lists the caller's enrolled courses with computed progress.

Auth: authenticated.

Success — **200** — `{ "courses": [...] }`, newest `subscribed_at` first.

Each entry (`mapEnrolledCourse` plus `progress`, **handler**):

Field	Type	Notes
<code>enrollmentId</code>	uuid	
<code>enrolledAt</code>	timestampz	<code>subscribed_at</code>
<code>id</code>	uuid	Course id
<code>code</code>	string	<code>course_code</code> , falling back to <code>id</code>
<code>title</code>	string	
<code>summary</code>	string	
<code>description</code>	string	
<code>totalLearningTime</code>	string	
<code>coverUrl</code>	string	
<code>price</code>	number	THB
<code>lessonCount</code>	integer	

Field	Type	Notes
<code>progress</code>	number	0–100, clamped

```
{
  "courses": [
    {
      "enrollmentId": "b71d5e90-3a2c-4f18-9e6d-0c8a4b217e33",
      "enrolledAt": "2026-08-14T09:21:03.512Z",
      "id": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34",
      "code": "MKT101",
      "title": "Digital Marketing Fundamentals",
      "summary": "Build a practical foundation in channels, funnels, and campaign measurement.",
      "description": "Learn how to plan, launch, and measure digital marketing campaigns...",
      "totalLearningTime": "10",
      "coverUrl": "/courses/service-design.svg",
      "price": 4590,
      "lessonCount": 3,
      "progress": 33
    }
  ]
}
```

Errors: `401 Unauthorized`; `500 {"error":"Failed to load enrolled courses"}`.

POST /api/enrollments

Enrolls the caller in a course. Idempotent.

Auth: authenticated. Note this endpoint performs **no payment check** — see [Known gaps](#).

Request body (handler)

Field	Type	Required	Description
<code>courseId</code>	uuid	Yes	Must resolve to an existing <code>courses.id</code>

```
{ "courseId": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34" }
```

Success

Status	When
<code>201</code>	A new enrollment row was created (<code>already: false</code>)
<code>200</code>	The caller was already enrolled (<code>already: true</code>)

```
{ "ok": true, "already": false, "id": "b71d5e90-3a2c-4f18-9e6d-0c8a4b217e33" }
```

Errors

Status	Body	Trigger
400	<code>{"error": "Invalid JSON body"}</code>	Unparseable body
400	<code>{"error": "Course id is required"}</code>	Missing/blank <code>courseId</code>
401	<code>{"error": "Unauthorized"}</code>	No session
404	<code>{"error": "Course not found"}</code>	No course with that id
500	<code>{"error": "Failed to subscribe to this course"}</code>	Insert failed

3.5 Wishlist

Unlike other course endpoints, wishlist accepts **either a course UUID or a course code** (matched case-insensitively against `course_code`).

GET /api/wishlist?count=1

Returns the caller's wishlist size. `count=1` is mandatory — this endpoint has no list mode.

Auth: authenticated.

Name	Type	Required	Description
<code>count</code>	string	Yes	Must be exactly <code>"1"</code> ; anything else returns 400

Success — 200

```
{ "count": 3 }
```

Errors: 400 `{"error": "Unsupported wishlist request"}` when `count` is not `"1"`; 401 Unauthorized; 500 `{"error": "Failed to load wishlist count"}`.

POST /api/wishlist

Adds a course to the wishlist. Idempotent, and refuses courses the caller already owns.

Auth: authenticated.

Field	Type	Required	Description
courseId	string	Yes	Course UUID or course code

```
{ "courseId": "MKT101" }
```

Success

Status	When	Body
201	Newly added	<code>{"ok":true,"already":false,"id":"<uuid>"}</code>
200	Already on the wishlist	<code>{"ok":true,"already":true,"id":"<uuid>"}</code>
200	Unique-constraint race	<code>{"ok":true,"already":true}</code> (no id)

Errors

Status	Body	Trigger
400	<code>{"error":"Invalid JSON body"}</code>	Unparseable body
400	<code>{"error":"Course id is required"}</code>	Missing <code>courseId</code>
400	<code>{"error":"You already own this course and cannot add it to your wishlist"}</code>	Caller is enrolled
401	<code>{"error":"Unauthorized"}</code>	No session
404	<code>{"error":"Course not found"}</code>	Neither id nor code matched
500	Various <code>Failed to ...</code>	Query failures

DELETE /api/wishlist

Removes a course from the wishlist. Idempotent — deleting an absent row still returns 200.

Auth: authenticated.

The course id is taken from the `courseId` **query param** first; if absent, the handler falls back to a JSON body with `courseId`. Both accept a UUID or a course code.

```
DELETE /api/wishlist?courseId=MKT101
```

Success — 200

```
{ "ok": true }
```

Errors: 400 {"error":"Course id is required"} when neither source supplies one; 401 Unauthorized ; 500 {"error":"Failed to remove from wishlist"}.

3.6 Progress

POST /api/progress

Records a learner's progress event against a sub-lesson.

Auth: authenticated **and enrolled in the course.**

Request body (handler)

Field	Type	Required	Description
courseId	uuid	Yes	
subLessonId	uuid	Yes	Must belong to courseId
action	string	Yes	Exactly one of visit , complete , submit_assignment

```
{
  "courseId": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34",
  "subLessonId": "d4e6a812-7c05-49b3-8f21-3a9e0d5c6b74",
  "action": "complete"
}
```

Success

Status	When
201	A new progress row was created
200	An existing row was updated

```
{ "ok": true, "id": "5a1c8f37-9e42-4d06-b83a-1f7c25e0a9d8", "created": false }
```

Errors

Status	Body	Trigger
400	{"error":"Invalid JSON body"}	Unparseable body

Status	Body	Trigger
400	<code>{"error": "Course id is required"}</code>	Missing <code>courseId</code>
400	<code>{"error": "Sub-lesson id is required"}</code>	Missing <code>subLessonId</code>
400	<code>{"error": "Invalid progress action"}</code>	<code>action</code> outside the allowed set
401	<code>{"error": "Unauthorized"}</code>	No session
403	<code>{"error": "You must be enrolled in this course"}</code>	Not enrolled
404	<code>{"error": "Sub-lesson not found"}</code>	Sub-lesson missing, or not in that course
500	<code>{"error": "Failed to save progress"}</code>	Write failed

3.7 Assignment submissions

Both endpoints resolve access through `getEnrolledAssignment()` (`src/lib/student-assignment-access.js`), which returns these statuses:

Status	Message	Trigger
400	<code>Assignment id is required</code>	Blank id
404	<code>Assignment not found</code>	No such assignment
403	<code>Forbidden</code>	Caller is not enrolled in the assignment's course
500	<code>Failed to load assignment</code>	Query error

PUT `/api/assignments/{id}/submission`

Creates or replaces the caller's submission. There is one submission per (assignment, user) — the unique index enforces it and the handler upserts.

Auth: authenticated and enrolled.

Path params: `id` (uuid) — the assignment id.

Request body

Field	Type	Required	Description
<code>content</code>	string	Yes	Meaning depends on the assignment's <code>submission_type</code>

`content` is validated by `validateStudentSubmissionContent()` (**schema**):

submission_type	content must be
text	Non-empty text
url	A valid <code>http: / https:</code> URL
choice	Comma-separated letters from <code>A,B,C,D</code> ; canonicalised to sorted order (e.g. <code>"B,A"</code> → <code>"A,B"</code>)
file	A storage path <code>"{userId}/{assignmentId}/{filename}"</code> , from the file endpoint below. Rejects <code>.</code> / <code>..</code> segments and nested paths.

```
{ "content": "A,C" }
```

Success

Status	When
201	First submission
200	Replaced an existing one

Field	Type	Notes
ok	boolean	
content	string	The canonicalised, stored value
submittedAt	timestamp _{tz}	
status	string	Always <code>"submitted"</code>
answerText	string	Only for <code>text</code> assignments with an answer key set
correctChoice	string	Only for <code>choice</code> assignments with a valid key

The answer key is returned **after** submitting, so the learner can self-check.

```
{
  "ok": true,
  "content": "A,C",
  "submittedAt": "2026-09-07T04:15:22.108Z",
  "status": "submitted",
  "correctChoice": "A,C"
}
```

Errors: the access-control table above, plus `400 {"error": "Invalid JSON body"}`, `400` with the validation message (`Please fill out this field`, `Enter a valid URL.`, `Invalid file path.`), `401 Unauthorized`, and `500` on write failure.

POST /api/assignments/{id}/submission/file

Uploads a file for a `file`-type assignment into the `assignment-submissions` bucket. Returns a path — it does **not** create the submission. Call `PUT ../submission` with the returned `path` as `content` afterwards.

Auth: authenticated and enrolled.

Path params: `id` (uuid) — the assignment id.

Request: `multipart/form-data`

Field	Type	Required	Rules
<code>file</code>	File	Yes	Must match the assignment's <code>allowed_file_types</code> and <code>max_file_size_mb</code>

Allowed kinds come from the assignment row (**schema** via `validateStudentUploadFile`):

Kind	Extensions	MIME types
<code>pdf</code>	<code>.pdf</code>	<code>application/pdf</code>
<code>doc</code>	<code>.doc</code> , <code>.docx</code>	<code>application/msword</code> , <code>application/vnd.openxmlformats-officedocument.wordprocessingml.document</code>
<code>image</code>	<code>.jpg</code> , <code>.jpeg</code> , <code>.png</code>	<code>image/jpeg</code> , <code>image/jpg</code> , <code>image/png</code>

Stored at `{userId}/{assignmentId}/{sanitizedName}` with `upsert: true`, so re-uploading the same filename overwrites.

Success — 200

```
{
  "path": "3c9a7e21-5d84-4f0b-8a16-7be2c4d09f55/9b3f7a02-6c14-4e88-a5d7-0e2b8c1f4a63/campaign-brief.pdf",
  "name": "campaign-brief.pdf"
}
```

Errors

Status	Body	Trigger
400	<code>{"error": "Expected multipart form data"}</code>	Not form data
400	<code>{"error": "This assignment does not accept file uploads."}</code>	<code>submission_type !== "file"</code>
400	<code>{"error": "Please fill out this field"}</code>	No file / zero bytes
400	<code>{"error": "Unsupported file type: <type>"}</code>	Kind not allowed
400	<code>{"error": "File exceeds max size of <n> MB"}</code>	Over the assignment's limit
401 / 403 / 404		Access-control table above
500	<code>{"error": "Upload failed"}</code>	Storage error

3.8 Promo codes

POST /api/promo-codes/validate

Validates a promo code against a course and returns the resulting total. **The subtotal comes from the database, never from the request** — the handler's own comment says so.

Auth: authenticated.

Request body (handler + schema constants)

Field	Type	Required	Rules
<code>code</code>	string	Yes	Must be a string, ≤ 64 chars (<code>PROMO_CODE_MAX_LENGTH</code>)
<code>courseId</code>	uuid	Yes	Must match the UUID pattern <code>PROMO_COURSE_ID_PATTERN</code>

```
{ "code": "MERRYX25", "courseId": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34" }
```

Success — 200

Field	Type	Description
<code>ok</code>	boolean	Always <code>true</code>

Field	Type	Description
<code>code</code>	string	Canonical promo code from the database
<code>discountType</code>	string	<code>fixed</code> or <code>percent</code>
<code>discountValue</code>	number	200 for a ฿200 code, 25 for a 25% code
<code>discountAmount</code>	number	THB actually deducted
<code>subtotal</code>	number	Course price from the database
<code>total</code>	number	<code>subtotal - discountAmount</code>
<code>minPurchaseAmount</code>	number	<code>0</code> when unset

```
{
  "ok": true,
  "code": "MERRYX25",
  "discountType": "percent",
  "discountValue": 25,
  "discountAmount": 1147.5,
  "subtotal": 4590,
  "total": 3442.5,
  "minPurchaseAmount": 1200
}
```

Errors

Status	Body	Trigger
400	<code>{"error": "Invalid JSON body"}</code> / <code>{"error": "Invalid promo request"}</code>	Unparseable, or not a plain object
400	<code>{"error": "Promo code must be at most 64 characters."}</code>	Non-string or over-long code
400	<code>{"error": "Promo code is required."}</code>	Empty after normalisation
400	<code>{"error": "A valid course id is required."}</code>	<code>courseId</code> is not a UUID
400	RPC message	Code expired, wrong course, or below <code>min_purchase_amount</code> (exact strings come from the <code>lookup_checkout_promo</code> RPC — see <i>Known gaps</i>)
401	<code>{"error": "Unauthorized"}</code>	No session

Status	Body	Trigger
429	RPC message	Database-side promo rate limit
503	<code>{"error": "Promo validation is temporarily unavailable."}</code>	RPC errored or returned nothing

3.9 Payments

Payments use Omise. Amounts are converted to satang (`toSatang`) before charging.

POST /api/payments/charge

Creates an Omise charge for a course and, for paid card charges, enrolls the buyer.

Auth: authenticated.

Request body (handler)

Field	Type	Required	Description
<code>paymentMethod</code>	string	Yes	<code>card</code> or <code>qr</code>
<code>courseId</code>	uuid	Yes	Must be a purchasable course
<code>omiseToken</code>	string	If <code>card</code>	Omise card token
<code>sourceId</code>	string	If <code>qr</code>	Omise PromptPay source id. <code>omiseSourceId</code> is accepted as an alias.
<code>promoCode</code>	string	No	Normalised; blank means no discount

```
{
  "paymentMethod": "card",
  "courseId": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34",
  "omiseToken": "tokn_test_placeholder000000",
  "promoCode": "NEWYEAR200"
}
```

Success — 200

Field	Type	Description
<code>ok</code>	boolean	For <code>card</code> , whether the charge is paid; for <code>qr</code> , whether a charge id exists
<code>status</code>	string	Omise charge status

Field	Type	Description
<code>paid</code>	boolean	
<code>enrolled</code>	boolean	Only ever <code>true</code> for a paid card charge
<code>alreadyEnrolled</code>	boolean	
<code>chargeId</code>	string	
<code>amount</code>	number	Final THB total after promo
<code>qrImage</code>	string null	PromptPay QR as a data URL; <code>null</code> for card
<code>expiresAt</code>	timestampz null	QR expiry, 5 minutes out; <code>null</code> for card

```
{
  "ok": true,
  "status": "successful",
  "paid": true,
  "enrolled": true,
  "alreadyEnrolled": false,
  "chargeId": "chrg_test_placeholder000000",
  "amount": 4390,
  "qrImage": null,
  "expiresAt": null
}
```

QR charges are not enrolled here — fulfilment happens via the webhook or the polling endpoint below.

Errors

Status	Body	Trigger
400	<code>{"error": "Invalid JSON body"}</code>	Unparseable body
400	<code>{"error": "Payment method must be card or qr."}</code>	Bad <code>paymentMethod</code>
400	<code>{"error": "Course id is required."}</code>	Missing <code>courseId</code>
400	<code>{"error": "Omise card token is required."}</code>	<code>card</code> without a token
400	<code>{"error": "Omise PromptPay source is required."}</code>	<code>qr</code> without a source
400	<code>{"error": "Amount is below the Omise minimum (20 THB)."} </code>	Total under 2000 satang

Status	Body	Trigger
400	Promo error, or Omise error message	Promo rejected, or charge creation failed
401	<code>{"error": "Unauthorized"}</code>	No session
403	<code>{"error": "This course is not available for purchase."}</code>	Course marked unavailable
404	<code>{"error": "Course not found."}</code>	Unknown course
429	Promo RPC message	Database-side promo rate limit
500	<code>{"error": "Failed to save payment."}</code> / <code>{"error": "Payment succeeded but enrollment failed."}</code>	Post-charge failures

GET /api/payments/charges/{id}

Polls an Omise charge and fulfils enrollment once it is paid. Used by the PromptPay "Refresh status" flow.

Auth: authenticated. Additionally, if the charge's `metadata.userId` is set and does not match the caller, the request is rejected with 403.

Path params: `id` (string) — the Omise charge id.

Success — 200

```
{
  "chargeId": "chrg_test_placeholder000000",
  "status": "successful",
  "paid": true,
  "enrolled": true,
  "already": false
}
```

When not yet paid, `paid`, `enrolled`, and `already` are all `false` and `status` reflects Omise.

Errors

Status	Body	Trigger
400	<code>{"error": "Charge id is required."}</code>	Blank id
400	<code>{"error": "Payment metadata is incomplete. Cannot enroll."}</code>	Charge metadata lacks the ids needed to fulfil
401	<code>{"error": "Unauthorized"}</code>	No session

Status	Body	Trigger
403	<code>{"error": "Forbidden"}</code>	Charge belongs to a different user
409	<code>{"error": "Payment received but enrollment is not complete yet. Try Refresh status.", "chargeId": "...", "status": "...", "paid": true, "enrolled": false}</code>	Paid but enrollment has not landed
500	<code>{"error": "Failed to load charge."}</code>	Omise lookup failed (status may come from the Omise error)

POST /api/payments/webhook

Omise webhook receiver. Re-fetches the event from Omise by id rather than trusting the posted body.

Auth: none in the handler. There is **no signature verification** — the safeguard is that the event is re-retrieved from Omise using only its id. Anyone who can guess a valid event id can trigger fulfilment for that event.

Request body: an Omise event object; only `id` and `object` are read from the request.

```
{ "id": "evnt_test_placeholder000000", "object": "event" }
```

Success — `200` in all handled cases, including ignored ones:

Body	When
<code>{"received": true, "ignored": true}</code>	Not an event object, unhandled charge shape, or nothing to fulfil
<code>{"received": true, "ignored": true, "key": "charge.expire"}</code>	Event key outside <code>charge.complete</code> / <code>charge.create</code>
<code>{"received": true, "pending": true, "chargeId": "chrg_..."}</code>	Charge not yet paid
<code>{"received": true, "chargeId": "chrg_...", "paid": true, "enrolled": true, "already": false}</code>	Fulfilled

Errors: `400 {"error": "Invalid JSON body"}`; `400` with the Omise message if event retrieval fails; `500 {"error": "Failed to process payment webhook."}`.

3.10 Admin — courses

All endpoints in 3.10–3.14 require **admin** (`requireAdmin()`): `401` without a session, `403` for a non-admin session.

GET /api/admin/courses

Paginated admin course list with search, tag filter, and status filter.

Query params

Name	Type	Required	Description
<code>page</code>	integer	No	Default <code>1</code> ; clamped to ≥ 1
<code>pageSize</code>	integer	No	Default <code>10</code> ; clamped to 1–50
<code>q</code>	string	No	Matches <code>title</code> or <code>course_code</code>
<code>sortBy</code>	string	No	<code>courseCode</code> (default), <code>title</code> , <code>price</code> , <code>createdAt</code> , <code>updatedAt</code>
<code>sortDirection</code>	string	No	<code>desc</code> for descending; anything else ascending
<code>status</code>	string	No	<code>all</code> (default), <code>active</code> , <code>inactive</code>
<code>tag</code>	string	No	Tag slug: <code>development</code> , <code>marketing</code> , <code>business</code> . <code>all</code> disables the filter. An unknown slug returns an empty page.

Success — `200`

```
{
  "courses": [
    {
      "id": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34",
      "title": "Digital Marketing Fundamentals",
      "course_code": "MKT101",
      "cover_file_url": "/courses/service-design.svg",
      "cover_file_type": null,
      "price": 4590,
      "lesson_count": 3,
      "tag": "marketing",
      "tag_name": "Marketing",
      "is_active": true,
      "created_at": "2026-07-02T08:00:00.000Z",
      "updated_at": "2026-08-19T11:42:10.220Z"
    }
  ],
  "total": 8,
  "page": 1,
  "pageSize": 10
}
```

Errors: 401 , 403 , 429 (120/min per IP), 500 {"error":"Failed to load courses"} .

POST /api/admin/courses

Creates a course, optionally with nested lessons, a promo code, and a course-level attachment.

Request body (handler + validateCourseFields schema)

Field	Type	Required	Rules
title	string	Yes	Max 50 chars
courseCode	string	Yes	Max 32 chars, alphanumeric; must be unique (case-insensitive)
tag	string	Yes	development , marketing , or business . Defaults to development .
summary	string	Yes	Max 200 chars
description	string	Yes	Max 400 chars
price	number	Yes	Numeric

Field	Type	Required	Rules
<code>totalLearningTime</code>	number	Yes	Numeric; stored as a string
<code>coverImageUrl</code>	string	Yes	Non-blank; a storage path from the uploads endpoint
<code>videoTrailerUrl</code>	string	Yes	Non-blank
<code>lessons</code>	array	No	Each <code>{ title, sortOrder?, subLessons[] }</code>
<code>promo</code>	object	No	<code>{ code, discountType, discountValue, minPurchaseAmount? }</code>
<code>attachment</code>	object	No	<code>{ name, fileUrl, fileType }</code>

For `promo.discountType`, this endpoint accepts `"thb"` (mapped to `fixed`) or `"percent"`.

Each `lessons[].subLessons[]` entry: `{ title, description?, videoUrl?, videoName?, attachmentUrl?, attachmentName?, attachmentType? }`. Video materials are stored with `file_type: "video/mp4"`; attachments default to `application/pdf`.

```

{
  "title": "Digital Marketing Fundamentals",
  "courseCode": "MKT101",
  "tag": "marketing",
  "summary": "Build a practical foundation in channels, funnels, and campaign measurement.",
  "description": "Learn how to plan, launch, and measure digital marketing campaigns across search, social, email, and content.",
  "price": 4590,
  "totalLearningTime": 10,
  "coverImageUrl": "course-covers/3c9a7e21-.../cover.jpg",
  "videoTrailerUrl": "course-trailers/3c9a7e21-.../trailer.mp4",
  "lessons": [
    {
      "title": "Channels & Funnel Basics",
      "sortOrder": 1,
      "subLessons": [
        { "title": "Mapping the funnel", "description": "Awareness to conversion." }
      ]
    }
  ],
  "promo": {
    "code": "NEWYEAR200",
    "discountType": "thb",
    "discountValue": 200,
    "minPurchaseAmount": 0
  }
}

```

Success — 201

```
{ "id": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34" }
```

Errors

Status	Body	Trigger
400	<pre>{ "error": "Missing or invalid required course fields", "fields": { ... }, "required": [...] }</pre>	Any required field missing or invalid. <code>fields</code> maps field → message; <code>required</code> lists all nine required names.
400	<pre>{ "error": "Invalid promo payload", "fields": { ... } }</pre>	Promo block present but invalid
401 / 403		Not an admin

Status	Body	Trigger
409	<code>{"error": "Course code already exists.", "fields": {"courseCode": "Course code already exists."}}</code>	Duplicate course code
500	<code>Failed to create course / Failed to create promo / Failed to create lessons / Failed to create attachment</code>	Write failure. Promo, lesson, and attachment failures roll the course row back; individual sub-lesson failures are skipped silently.

On success `revalidateTag("courses")` is called so the public catalog cache refreshes.

GET /api/admin/courses/{id}

Fetches one course in the shape the admin edit form expects.

Path params: `id` (uuid).

Success — 200

```
{
  "id": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34",
  "title": "Digital Marketing Fundamentals",
  "courseCode": "MKT101",
  "tag": "marketing",
  "summary": "Build a practical foundation in channels, funnels, and campaign measurement.",
  "description": "Learn how to plan, launch, and measure digital marketing campaigns...",
  "price": 4590,
  "totalLearningTime": "10",
  "coverImageUrl": "course-covers/3c9a7e21-.../cover.jpg",
  "videoTrailerUrl": "course-trailers/3c9a7e21-.../trailer.mp4",
  "promo": {
    "id": "6d0b3f19-8c72-4a51-be94-27f0a5c81d63",
    "code": "NEWYEAR200",
    "discountType": "thb",
    "discountValue": 200,
    "minPurchaseAmount": 0
  },
  "attachment": {
    "id": "e271b940-5a38-4c6f-8d02-9b13c7e45a80",
    "name": "Campaign planning worksheet",
    "fileUrl": "course-attachments/3c9a7e21-.../worksheet.pdf",
    "fileType": "application/pdf"
  }
}
```

`promo` and `attachment` are `null` when absent. Note `discountType` is mapped **to the UI form** here: a stored `fixed` is returned as `"thb"`.

Errors: `400 {"error":"Course id is required"}; 401 / 403; 404 {"error":"Course not found"}; 500 {"error":"Failed to load course"}.`

PUT /api/admin/courses/{id}

Full update of a course, its promo, and its course-level attachment.

Path params: `id` (uuid).

Request body: same fields as `POST /api/admin/courses`, except:

- `lessons` is **not** processed — lessons are managed through the lesson endpoints.
- `promo.discountType` must be `"thb"` or `"percent"` here (the create endpoint also tolerated a raw `"fixed"`).
- Omitting `promo` **deletes** any existing promo for the course.

Success — 200

```
{ "id": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34", "success": true }
```

Errors: as `POST`, plus `404 {"error":"Course not found"}`. Replaced cover, trailer, and attachment objects are deleted from Storage after a successful update.

PATCH /api/admin/courses/{id}

Toggles a course's active flag. This is the only field this method touches.

Path params: `id` (uuid).

Field	Type	Required	Description
<code>isActive</code>	boolean	Yes	Must be a real boolean

```
{ "isActive": false }
```

Success — 200

```
{ "id": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34", "is_active": false, "success": true }
```

Errors: `400 {"error":"Invalid JSON body"}; 400 {"error":"isActive must be a boolean."}; 400 {"error":"Course id is required"}; 401 / 403; 404 {"error":"Course not found"}; 500 {"error":"Failed to update course status"}.`

DELETE /api/admin/courses/{id}

Deletes a course and its materials, sub-lessons, and lessons, then removes its Storage objects.

Path params: `id` (uuid).

Success — 200

```
{ "ok": true }
```

Errors: 400 {"error":"Course id is required"}; 401 / 403; 500 with Failed to delete course materials / ... sub-lessons / ... lessons / Failed to delete course depending on which stage failed.

GET /api/admin/course-tags

Lists the course tag vocabulary, ordered by name.

Success — 200

```
{
  "tags": [
    { "slug": "business", "name": "Business" },
    { "slug": "development", "name": "Development" },
    { "slug": "marketing", "name": "Marketing" }
  ]
}
```

These three rows are seeded by `docs/sql/020_course_tags.sql` and mirrored in `COURSE_TAG_OPTIONS`.

Errors: 401 / 403; 500 {"error":"Failed to load course tags"}.

3.11 Admin — lessons

Both lesson route files contain fallback behaviour that can silently act on the wrong row.

`resolveCourseId()` falls back to "the first course in the database" — and, if there are none, **creates a demo course** — when the supplied id is not a valid UUID or does not exist. `resolveLessonId()` similarly falls back to the first lesson, then to a hard-coded id. Flagged here because it materially changes what these endpoints do with a bad id; see [Known gaps](#).

GET /api/admin/courses/{id}/lessons

Lists a course's lessons with sub-lesson counts, ordered by `sort_order`.

Path params: `id` (uuid) — subject to the fallback above.

Success — 200

```
{
  "lessons": [
    { "id": "a5c81e07-3b64-4f29-9d17-6e0a2c8b5f41", "name": "Channels & Funnel Basics",
      "subLessons": 4, "sortOrder": 1 }
  ]
}
```

Returns `{"lessons": []}` when no course could be resolved.

Errors: 400 `{"error": "Course id is required"}`; 401 / 403; 500 `{"error": "Failed to load lessons"}`.

POST /api/admin/courses/{id}/lessons

Creates a lesson with its sub-lessons and their media. `sort_order` is assigned as `max(existing) + 1`.

Path params: `id` (uuid).

Field	Type	Required	Description
<code>lessonName</code>	string	Yes	Non-blank
<code>subLessons</code>	array	Yes	At least one; each needs a non-blank <code>title</code>

Sub-lesson fields: `title` (required), `description`, `videoUrl`, `videoName`, `attachmentUrl`, `attachmentName`, `attachmentType`.

```
{
  "lessonName": "Campaign Measurement",
  "subLessons": [
    {
      "title": "Reading a performance dashboard",
      "description": "Sessions, CAC, and ROAS.",
      "videoUrl": "course-videos/3c9a7e21-.../dashboard.mp4",
      "videoName": "Dashboard walkthrough"
    }
  ]
}
```

Success — 201

```
{ "success": true, "id": "a5c81e07-3b64-4f29-9d17-6e0a2c8b5f41" }
```

Errors

Status	Body	Trigger
400	<code>{"error": "Course id is required"}</code>	Missing path id
400	<code>{"error": "Invalid JSON body"}</code>	Unparseable body
400	<code>{"error": "Lesson name is required"}</code>	Blank <code>lessonName</code>
400	<code>{"error": "At least one sub-lesson is required"}</code>	Empty <code>subLessons</code>
400	<code>{"error": "Sub-lesson #2 name is required"}</code>	A sub-lesson has a blank title (1-based index)
401 / 403		Not an admin
404	<code>{"error": "Course not found and could not be resolved"}</code>	No course could be resolved or created
500	<code>{"error": "Failed to create lesson"}</code>	Lesson insert failed

Sub-lesson insert failures are skipped without failing the request, so a `201` does not guarantee every sub-lesson was created.

GET `/api/admin/courses/{id}/lessons/{lessonId}`

Fetches one lesson with its sub-lessons and their media, shaped for the edit form.

Path params: `id` (uuid), `lessonId` (uuid).

A material is classified as the video when its `file_type` starts with `video/`, its URL contains `course-trailers` or `trailer`, or its extension is `.mp4/.webm/.mov/.m4v`. The first non-video material becomes the attachment.

Success — `200`

```
{
  "lesson": {
    "id": "a5c81e07-3b64-4f29-9d17-6e0a2c8b5f41",
    "name": "Campaign Measurement",
    "subLessons": [
      {
        "id": "d4e6a812-7c05-49b3-8f21-3a9e0d5c6b74",
        "title": "Reading a performance dashboard",
        "description": "Sessions, CAC, and ROAS.",
        "videoUrl": "course-videos/3c9a7e21-.../dashboard.mp4",
        "videoName": "Dashboard walkthrough",
        "videoFile": null,
        "attachmentUrl": null,
        "attachmentName": "",
        "attachmentType": null,
        "attachmentFile": null
      }
    ]
  }
}
```

Returns `200 {"lesson": null}` — not a 404 — when the lesson cannot be found.

Errors: `400 {"error": "Course id and Lesson id are required"}`; `401` / `403`.

PUT /api/admin/courses/{id}/lessons/{lessonId}

Updates a lesson. **Sub-lessons are deleted and recreated**, so their ids change and any `sub_lesson_progress` rows referencing them cascade away.

Path params: `id` (uuid), `lessonId` (uuid).

Request body: identical to `POST .../lessons`.

Success — `200`

```
{ "success": true }
```

Errors: same 400s as `POST .../lessons`, plus `401` / `403`, `404 {"error": "Lesson not found"}`, and `500 {"error": "Failed to update lesson"}`. Orphaned media is cleaned from Storage afterwards.

DELETE /api/admin/courses/{id}/lessons/{lessonId}

Deletes a lesson and its sub-lessons, then cleans up their Storage objects.

Path params: `id` (uuid), `lessonId` (uuid).

```
{ "success": true }
```

Errors: 400 {"error":"Course id and Lesson id are required"} ; 401 / 403 ; 500 {"error":"Failed to delete lesson sub-lessons"} or {"error":"Failed to delete lesson"} .

3.12 Admin — assignments

GET /api/admin/assignments

Lists assignments. **Filtering, sorting, and pagination happen in memory** — the handler selects all assignments, then `processAdminAssignments()` applies the parameters.

Query params

Name	Type	Required	Description
page	integer	No	Default 1
pageSize	integer	No	Default 10 ; clamped to 1–50
q	string	No	Free-text search
status	string	No	Default all
sortBy	string	No	Default updatedAt
sortDirection	string	No	asc , else desc

Success — 200 — the spread of `processAdminAssignments()` plus `page` and `pageSize` . Each row (`mapAdminAssignment` , **handler**):

Field	Type	Notes
id	uuid	
title	string	
description	string	"" when null
courseTitle	string	"-" when unlinked
lessonTitle	string	"-" when unlinked
subLessonTitle	string	"-" when unlinked
createdDateLabel	string	Formatted for display; "-" when null
updatedDateLabel	string	Formatted for display; "-" when null
is_active	boolean	

Field	Type	Notes
<code>created_at</code>	timestamptz null	
<code>updated_at</code>	timestamptz null	

The sibling keys alongside `page` / `pageSize` come from `processAdminAssignments()` (not read in full — see *Known gaps*).

```
{
  "assignments": [
    {
      "id": "9b3f7a02-6c14-4e88-a5d7-0e2b8c1f4a63",
      "title": "Draft a channel plan",
      "description": "Pick two channels and justify the split.",
      "courseTitle": "Digital Marketing Fundamentals",
      "lessonTitle": "Channels & Funnel Basics",
      "subLessonTitle": "Mapping the funnel",
      "createdDateLabel": "12/08/2026",
      "updatedDateLabel": "19/08/2026",
      "is_active": true,
      "created_at": "2026-08-12T10:04:55.000Z",
      "updated_at": "2026-08-19T11:42:10.220Z"
    }
  ],
  "total": 12,
  "page": 1,
  "pageSize": 10
}
```

Errors: `401` / `403` ; `429` (120/min per IP); `500 {"error":"Failed to load assignments"}`.

POST /api/admin/assignments

Creates an assignment.

Request body (handler + `assignmentAnswerColumns` schema)

Field	Type	Required	Rules
<code>courseId</code>	uuid	Yes	
<code>lessonId</code>	uuid	Yes	Validated as present, but not stored — the link is via <code>subLessonId</code>
<code>subLessonId</code>	uuid	Yes	

Field	Type	Required	Rules
title	string	Yes	Non-blank
description	string	No	null when blank
submissionType	string	No	text (default), file, url, choice
allowedFileTypes	string[]	If file	At least one of pdf, doc, image
maxFileSizeMb	number	If file	Exactly one of 5, 10, 20, 50
answerText	string	If text	Non-blank answer key
choiceA – choiceD	string	If choice	All four required
correctChoice	string	If choice	Comma-separated letters from A, B, C, D; multiple allowed, canonicalised

```
{
  "courseId": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34",
  "lessonId": "a5c81e07-3b64-4f29-9d17-6e0a2c8b5f41",
  "subLessonId": "d4e6a812-7c05-49b3-8f21-3a9e0d5c6b74",
  "title": "Identify the funnel stage",
  "submissionType": "choice",
  "choiceA": "Awareness",
  "choiceB": "Consideration",
  "choiceC": "Conversion",
  "choiceD": "Retention",
  "correctChoice": "B,C"
}
```

Success — 201

```
{ "success": true, "id": "9b3f7a02-6c14-4e88-a5d7-0e2b8c1f4a63" }
```

Errors

Status	Body	Trigger
400	{"error": "Invalid JSON body"}	Unparseable body
400	{"error": "Course, lesson, and sub-lesson are required"}	Any of the three ids blank
400	{"error": "Assignment title is required"}	Blank title

Status	Body	Trigger
400	<code>{"error":"Invalid submission type."}</code>	Outside the four types
400	<code>{"error":"Select at least one allowed file type."}</code>	<code>file</code> type with no valid kinds
400	<code>{"error":"Select a valid max file size."}</code>	<code>maxFileSizeMb</code> not 5/10/20/50
400	<code>{"error":"Please fill out this field"}</code>	Missing answer key for <code>text</code> or <code>choice</code>
401 / 403		Not an admin
429	<code>{"error":"Too many assignment creates, try again in a moment"}</code>	20 per 15 min per admin
500	<code>{"error":"Failed to create assignment"}</code>	Insert failed

GET /api/admin/assignments/{id}

Fetches one assignment for the edit form. `lessonId` is derived from the sub-lesson.

Path params: `id` (uuid).

Success — `200`

```
{
  "assignment": {
    "id": "9b3f7a02-6c14-4e88-a5d7-0e2b8c1f4a63",
    "courseId": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34",
    "lessonId": "a5c81e07-3b64-4f29-9d17-6e0a2c8b5f41",
    "subLessonId": "d4e6a812-7c05-49b3-8f21-3a9e0d5c6b74",
    "title": "Identify the funnel stage",
    "description": "",
    "submissionType": "choice",
    "allowedFileTypes": [],
    "maxFileSizeMb": 20,
    "answerText": "",
    "choiceA": "Awareness",
    "choiceB": "Consideration",
    "choiceC": "Conversion",
    "choiceD": "Retention",
    "correctChoice": "B,C"
  }
}
```

`maxFileSizeMb` defaults to `20` when null. Answer-key fields are always present, as empty strings when unset.

Errors: `400 {"error":"Assignment id is required"}`; `401 / 403`; `404 {"error":"Assignment not found"}` — also returned when the query itself errors.

PATCH /api/admin/assignments/{id}

Two distinct modes, chosen by whether the body has an `isActive` property.

Path params: `id` (uuid).

Mode 1 — status toggle. Triggered when `isActive` is present.

```
{ "isActive": false }
```

```
{
  "id": "9b3f7a02-6c14-4e88-a5d7-0e2b8c1f4a63",
  "is_active": false,
  "updated_at": "2026-09-07T04:20:11.900Z",
  "success": true
}
```

Mode 2 — full update. Triggered when `isActive` is absent. Body and validation are identical to `POST /api/admin/assignments` (no create rate limit).

```
{ "success": true, "id": "9b3f7a02-6c14-4e88-a5d7-0e2b8c1f4a63" }
```

Errors: 400 {"error": "isActive must be a boolean."} when `isActive` is present but not boolean; otherwise the POST error table; 404 {"error": "Assignment not found"}; 500 {"error": "Failed to update assignment"} or {"error": "Failed to update assignment status"}.

DELETE /api/admin/assignments/{id}

Deletes an assignment. Submissions cascade via the foreign key.

Path params: `id` (uuid).

Success — 200

```
{ "success": true }
```

Errors: 400 {"error": "Assignment id is required"}; 401 / 403; 500 {"error": "Failed to delete assignment"}. Deleting a non-existent id still returns 200.

3.13 Admin — promo codes

GET /api/admin/promo-codes

Paginated promo code list. Searching for the literal string `all` (case-insensitive) is special-cased to mean "promos that apply to every course" — those with no `course_id` and no course links.

Query params

Name	Type	Required	Description
<code>page</code>	integer	No	Default <code>1</code>
<code>pageSize</code>	integer	No	Default <code>10</code> ; clamped to 1–50
<code>q</code>	string	No	Matches <code>code</code> , <code>discount_type</code> , or a linked course code. <code>all</code> switches to the all-courses filter.
<code>sortBy</code>	string	No	<code>code</code> (default), <code>minPurchase</code> , <code>discountValue</code> , <code>createdAt</code> , <code>updatedAt</code>
<code>sortDirection</code>	string	No	<code>desc</code> for descending; anything else ascending

```
{
  "promoCodes": [
    {
      "id": "6d0b3f19-8c72-4a51-be94-27f0a5c81d63",
      "code": "MERRYX25",
      "discount_type": "percent",
      "discount_value": 25,
      "min_purchase_amount": 1200,
      "course_id": null,
      "starts_at": "2022-12-02T15:30:00.000Z",
      "updated_at": "2026-08-19T11:42:10.220Z",
      "promo_code_courses": [],
      "courseCodes": [],
      "appliesToAllCourses": true
    }
  ],
  "total": 5,
  "page": 1,
  "pageSize": 10
}
```

Rows are the raw database columns spread as-is, plus the derived `courseCodes` and `appliesToAllCourses`. The raw `promo_code_courses` join array is passed through.

Errors: 401 / 403 ; 429 (120/min per IP); 500 {"error": "Failed to load promo codes"} or {"error": "Failed to search promo codes"}.

POST /api/admin/promo-codes

Creates a promo code and its course links in one transaction via the `save_admin_promo` RPC.

Request body — validated by `normalizePromoInput()` (schema)

Field	Type	Required	Rules
<code>code</code>	string	Yes	1–64 letters/digits only (<code>/^[a-z0-9]+\$ /i</code>). Upper-cased.
<code>discountType</code>	string	Yes	<code>percent</code> , or <code>thb / fixed</code> (both stored as <code>fixed</code>)
<code>discountValue</code>	number	Yes	≥ 0 . For <code>percent</code> , ≤ 100 .
<code>minPurchaseAmount</code>	integer	Yes	Must be a non-negative integer

Field	Type	Required	Rules
courseIds	uuid[]	No	Defaults to [] (all courses). Every entry must be a valid UUID. Deduplicated.

There is a business rule for fixed discounts: `minPurchaseAmount` must be at least `discountValue + 100`, so the customer still pays $\geq$ \$100 (`MINIMUM_CUSTOMER_PAYMENT`).

```
{
  "code": "NEWMEMBER",
  "discountType": "thb",
  "discountValue": 300,
  "minPurchaseAmount": 3000,
  "courseIds": ["8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34"]
}
```

Success — 201

```
{ "id": "6d0b3f19-8c72-4a51-be94-27f0a5c81d63" }
```

Errors — the 400s come from `normalizePromoInput` ; the rest are mapped from Postgres error codes in `saveAdminPromo` :

Status	Body	Trigger
400	<code>{"error": "Invalid JSON body"}</code> / <code>{"error": "Invalid promo request."}</code>	Unparseable, or not a plain object
400	<code>{"error": "Promo code must contain 1-64 letters or numbers."}</code>	Bad code format
400	<code>{"error": "Select valid courses for this promo code."}</code>	<code>courseIds</code> not an array of UUIDs
400	<code>{"error": "Minimum purchase amount must be a non-negative number."}</code>	Not a non-negative integer
400	<code>{"error": "Invalid discount type."}</code>	Not <code>fixed</code> / <code>percent</code> after mapping
400	<code>{"error": "Discount value must be a non-negative number."}</code>	Negative or non-finite
400	<code>{"error": "Percentage discount cannot exceed 100%."}</code>	<code>percent</code> over 100

Status	Body	Trigger
400	<code>{"error": "Minimum purchase amount must be at least 400 THB for this discount."}</code>	Fixed discount below the \$100 floor rule
400	<code>{"error": "Invalid promo code or selected courses."}</code>	PG 23503 , 22023 , 22P02
401 / 403	<code>{"error": "Forbidden"}</code>	Not an admin, or PG 42501
409	<code>{"error": "Promo code already exists."}</code>	PG 23505
503	<code>{"error": "Unable to save promo code. Please try again."}</code>	Any other RPC failure

GET /api/admin/promo-codes/{id}

Fetches one promo code with its linked course ids.

Path params: `id` (uuid).

Success — `200` — raw columns plus a derived `courseIds`, which falls back to `[course_id]` when there are no join rows, or `[]` when both are absent.

```
{
  "id": "6d0b3f19-8c72-4a51-be94-27f0a5c81d63",
  "code": "NEWMEMBER",
  "discount_type": "fixed",
  "discount_value": 300,
  "min_purchase_amount": 3000,
  "course_id": null,
  "is_active": true,
  "starts_at": "2022-12-02T15:30:00.000Z",
  "updated_at": "2026-08-19T11:42:10.220Z",
  "promo_code_courses": [
    { "course_id": "8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34", "courses": { "course_code": "MKT101" } }
  ],
  "courseIds": ["8f2b1c44-0a3e-4d61-9b7c-2e5a9d0c1f34"]
}
```

Errors: `401` / `403`; `404` with the raw Postgres message as `error` — this handler maps **any** query error to 404, including genuine failures.

PATCH /api/admin/promo-codes/{id}

Two modes, chosen by whether `isActive` is a boolean.

Path params: `id` (uuid).

Mode 1 — status toggle. When `isActive` is a boolean:

```
{ "isActive": false }
```

```
{ "id": "6d0b3f19-8c72-4a51-be94-27f0a5c81d63", "is_active": false, "success": true }
```

Mode 2 — full update. Otherwise the body is validated by `normalizePromoInput()` exactly as in `POST`, and saved through the same RPC.

```
{ "id": "6d0b3f19-8c72-4a51-be94-27f0a5c81d63" }
```

Errors: the `POST` table, plus `404 {"error":"Promo code not found."}` (PG `P0002`, or a status toggle matching no row) and `500 {"error":"Failed to update promo code status."}`.

DELETE /api/admin/promo-codes/{id}

Deletes a promo code.

Path params: `id` (uuid).

Success — `200`

```
{ "ok": true }
```

Errors: `401` / `403`; `500` with the raw Postgres message.

3.14 Admin — uploads

POST /api/admin/uploads

Uploads course media to Supabase Storage and returns a reference to store on a course, lesson, or sub-lesson.

This path is **excluded from the proxy matcher**, so it is protected solely by `requireAdmin()` inside the handler.

Request: `multipart/form-data`

Field	Type	Required	Description
<code>kind</code>	string	Yes	<code>cover</code> , <code>trailer</code> , <code>lessonVideo</code> , or <code>attachment</code>
<code>file</code>	File	Yes	Must satisfy the kind's limits

Limits from `UPLOAD_KINDS` (schema):

kind	Bucket	Max size	Accepted MIME types
<code>cover</code>	<code>course-covers</code>	5 MB	<code>image/jpeg</code> , <code>image/png</code> , <code>image/jpg</code>
<code>trailer</code>	<code>course-trailers</code>	20 MB	<code>video/mp4</code> , <code>video/quicktime</code> , <code>video/x-msvideo</code> , <code>video/avi</code>
<code>lessonVideo</code>	<code>course-videos</code> (private)	20 MB	Same as <code>trailer</code>
<code>attachment</code>	<code>course-attachments</code>	20 MB	Any — no MIME restriction

Stored at `{adminUserId}/{uuid}.{ext}` with `upsert: false`.

Success — 200

```
{
  "bucket": "course-covers",
  "path": "3c9a7e21-5d84-4f0b-8a16-7be2c4d09f55/7f1e2d93-4a60-4c85-9b32-8d05e6a71c49.jpg",
  "fileUrl": "course-covers/3c9a7e21-5d84-4f0b-8a16-7be2c4d09f55/7f1e2d93-4a60-4c85-9b32-8d05e6a71c49.jpg",
  "fileType": "image/jpeg",
  "name": "mkt101-cover.jpg"
}
```

`fileUrl` is the value to send back as `coverImageUrl`, `videoTrailerUrl`, `videoUrl`, or `attachment.fileUrl`.

Errors

Status	Body	Trigger
400	<code>{"error": "Expected multipart form data"}</code>	Not form data
400	<code>{"error": "Invalid kind. Use one of \"cover\", \"trailer\", \"lessonVideo\", \"attachment\"."}</code>	Unknown <code>kind</code>
400	<code>{"error": "File is required"}</code>	Missing/empty file

Status	Body	Trigger
400	<code>{"error": "File exceeds max size of 5 MB"}</code>	Over the kind's limit
400	<code>{"error": "Unsupported file type: video/x-flv"}</code>	MIME not allowed for that kind
401 / 403		Not an admin
500	<code>{"error": "Upload failed"}</code>	Storage error

GET /api/admin/uploads/signed-url

Signs a private lesson video so the admin editor can preview it. Only the `course-videos` bucket is private; every other media kind is public.

Query params

Name	Type	Required	Description
<code>path</code>	string	Yes	A stored <code>fileUrl</code> , e.g. <code>course-videos/{adminId}/{uuid}.mp4</code>

Success — 200

```
{ "signedUrl": "https://<project>.supabase.co/storage/v1/object/sign/course-videos/3c9a7e21-.../dashboard.mp4?token=<placeholder>" }
```

Errors: 400 `{"error": "Missing path"}` when `path` is absent or blank; 401 / 403 ; 404 `{"error": "Unable to sign media"}` when signing fails.

4. Server actions

This project has none.

Searching the entire repository (excluding `node_modules` and `.next`, case-insensitive, across `.js`, `.jsx`, `.ts`, `.tsx`, `.mjs`) for the `use server` directive returns zero matches. There are no `"use server"` files and no inline server-action functions.

All mutations therefore go over HTTP to the route handlers documented above. Client components call them with `fetch` — for example `loadMyCourses()` in `src/lib/enrollments.js` does `fetch("/api/enrollments")`.

For contrast, the codebase does use `"use client"` extensively, and server components read data directly through the Supabase server client rather than through this API.

If server actions are added later, the shape to document would be: exported async function name, file, auth requirement, argument types, return shape, error behaviour, and the fact that they are **invoked from components rather than over HTTP**. That section is intentionally empty rather than speculative.

5. Common error shapes

Every JSON error in this API has the same envelope, produced by `jsonError`:

```
{ "error": "Human-readable message" }
```

Some errors add sibling keys alongside `error`:

Extra key	Type	Used by	Meaning
<code>errors</code>	object	POST <code>/api/auth/register</code> , PATCH <code>/api/profile</code>	Field name → message. Valid fields map to <code>"</code> .
<code>fields</code>	object	Admin course create/update	Field name → message, for invalid fields only
<code>required</code>	string[]	Admin course create/update	Every field name the endpoint requires
<code>chargeId</code> , <code>status</code> , <code>paid</code> , <code>enrolled</code>	mixed	GET <code>/api/payments/charges/ {id}</code> 409	Charge state alongside the error

Success responses have **no** envelope — the payload sits at the top level. There is no global `success` flag: some endpoints return `{"ok":true}`, others `{"success":true}`, others the resource itself. This is inconsistent across the API and is noted in [Known gaps](#).

Status codes in this project

Status	Meaning here
200	Success. Also used for idempotent no-ops (already enrolled, already wishlisted) and for some "not found" cases that return <code>null</code> instead of a 404.
201	A row was created: enrollment, wishlist entry, progress row, first submission, course, lesson, assignment, promo code.
307	Redirect. Proxy admin guard, and <code>GET /auth/confirm</code> .
400	Validation failure, unparseable JSON, wrong content type, or a business rule (already own the course, amount below the Omise minimum).
401	No session. Body is always <code>{"error":"Unauthorized"}</code> .
403	Authenticated but not permitted: not an admin, not enrolled, course unavailable, charge belongs to another user. Body is usually <code>{"error":"Forbidden"}</code> .

Status	Meaning here
404	Resource does not exist. Also returned by <code>POST /api/auth/forgot-password</code> for an unregistered email, and by <code>GET /api/admin/promo-codes/{id}</code> for any query error.
409	Conflict: duplicate course code, duplicate promo code, or a paid charge whose enrollment has not completed.
429	Rate limited. Always carries a <code>Retry-After</code> header in seconds.
500	Server or database failure. The message is usually the underlying error's text, so it can leak database wording to the client.
503	The promo RPC was unavailable, or <code>save_admin_promo</code> failed for an unmapped reason.

6. Known gaps

Everything below is either unverified or a behaviour worth knowing before relying on this document.

Shapes not fully resolved

1. **processAdminAssignments() return keys.** `GET /api/admin/assignments` spreads this function's result. The per-row shape is confirmed via `mapAdminAssignment`, but the sibling keys next to `page / pageSize` come from `src/lib/admin-assignment-list.js`, which I did not read. The example shows `assignments` and `total` as the plausible keys — **treat those two names as inferred**.
2. **lookup_checkout_promo RPC error strings.** Promo validation and payment errors are passed through from this Postgres function. The SQL is in `docs/sql/025_promo_code_security.sql`, which I did not read line by line, so the exact user-facing strings for "expired", "wrong course", and "below minimum" are not enumerated here.
3. **save_admin_promo RPC.** Only its error codes are documented, mapped from `saveAdminPromo`. Its internal validation is not covered.
4. **Omise payloads.** `charge.status` values, `metadata` contents, and the event keys beyond `charge.complete / charge.create` come from Omise, not this codebase. Only what the handlers read is documented.
5. **Progress row shape.** `recordSubLessonProgress()` was not read; the endpoint's `{ ok, id, created }` response is confirmed from the handler, but which columns get written for each `action` is not.

Behaviours worth flagging

6. **POST /api/enrollments has no payment check.** Any authenticated user can enroll in any existing course by posting its id, regardless of price or payment history. The paid path (`/api/payments/charge`) enrolls separately. Whether RLS constrains this is not something I verified.
7. **Lesson endpoints silently substitute a different course or lesson.** As noted in 3.11, `resolveCourseId()` falls back to the first course in the database and will **create a demo course titled "Service Design Essentials"** if none exists; `resolveLessonId()` falls back to the first lesson, then to the hard-coded id `00000000-0000-0000-0000-000000000001`. A typo'd id can therefore mutate an unrelated row rather than returning 404.
8. **PUT ../lessons/{lessonId} destroys sub-lesson ids.** Sub-lessons are deleted and recreated on every update, so learner `sub_lesson_progress` rows cascade away.
9. **The payment webhook has no signature verification.** It re-fetches the event from Omise by id, which limits forgery, but there is no HMAC check.
10. **Forgot-password confirms account existence** (404 for unknown emails) and its user lookup only pages through the first ~4,000 users.
11. **Rate limiting is per-process and in-memory.** It resets on restart and does not work across replicas — the module says so itself.
12. **500 messages often echo the database error.** Many handlers use `error.message || "Fallback"`, so Postgres wording can reach the client.
13. **Success-shape inconsistency.** `{"ok":true}`, `{"success":true}`, and bare resource objects are all used. Grouped by resource above, but there is no single convention.

14. `GET /api/admin/promo-codes/{id}` maps every query error to **404**, so a genuine database failure is indistinguishable from a missing row.
15. **Admin assignment list filters in memory.** It selects all assignments before paginating, so it will degrade as the table grows.

Not covered

16. Supabase Auth's own endpoints (sign-in, sign-out, password update) are called from client components via `@supabase/supabase-js` and are not part of this API surface.
17. RLS policies are named as a second enforcement layer but not enumerated per table; see `docs/sql/` for the authoritative policies.